TOBB ETÜ Bilgisayar Mühendisliği Bölümü
YAP 441- BİL 541 Dönem Projesi Ara Rapor

2025-26 Bahar Dönemi

Adı Soyadı:ÖMER KARA **Öğrenci No:241401017** **Date: 02/03/2026**

1. Proje ismi

Entropi algoritması kullanarak wordle çözen yapay zeka

2. Proje tanımı [1] (kısa özet, problemin açıklanması, beklenen sonuçlar)

Projedeki amacım, Wordle oyununu oynayan ve mümkün olan en az tahminde doğru kelimeyi bulan deterministik bir yapay zeka modeli geliştirmektir. Wordle oyunu, 5 harfli bir kelimenin en fazla 6 tahmin içerisinde bulunmasını gerektiren bir bilgi çıkarım problemidir. Her tahmin sonrası oyuncuya üç tür geri bildirim verilir:

- **G (Green)**: Harf doğru ve doğru pozisyonda
- **Y (Yellow)**: Harf doğru fakat yanlış pozisyonda
- **B (Black/Gray)**: Harf kelimede bulunmuyor

Bu geri bildirimler, çözüm uzayını daraltmak için kullanılabilecek kısıt bilgisi üretir. Problem, bu kısıt bilgilerini en verimli şekilde kullanarak arama uzayını mümkün olan en hızlı biçimde daraltmaktır. Geliştireceğim model, machine learning kullanmayacak, bunun yerine bilgi teorisine dayanan deterministik modeller tercih edilecek. Böylece model, eğitim verisine ihtiyaç duymadan doğrudan algoritmik optimizasyon ile çözüm üretmektedir

Kelime tahmin algoritması bu projenin en önemli kısmı, çünkü yanlış bir algoritma tercihinin doğru sonucu vermesi yüzlerce deneme gerektirebilir. Bu alanda yapılmış diğer projeleri ve araştırmaları incelediğimde kelime seçmek için en uygun yöntemin entropi algoritması olduğuna karar verdim. Projemde entropi algoritmasını kullandım. Henüz projeyi tam olarak bitirmediğimden modelimin nasıl bir sonuç vereceğini bilmiyorum ama yaptığım araştırmalar bana entropi algoritmasının ortalama 3.7 denemede, doğru sonucu verdiğini gösteriyor. En kötü durumlarda deneme sayısı 6ya ancak çıkıyor. Dolayısıyla benim de proje sonucu için beklentim bu yönde. Kullandığım entropi algoritmasından kısaca bahsetmek gerekirse;
Bu algoritma, tahmin edilecek kelimelerin her biri için eldeki bilgileri kullanarak dönmesi beklenen desenleri (pattern) hesaplıyor. Aynı desene sahip olan kelimeler gruplanıyor (bucket). Bu grupların Shannon entropi değeri hesaplanıyor. Entropi değeri ne kadar yüksekse, tahmin ortalama olarak o kadar fazla bilgi üretir. En yüksek entropiye sahip tahmin seçiliyor.

3. Proje metodolojisi, daha önce benzer projelerde kullanılan metotlar hakkında bilgiler, literatur taraması

Geliştireceğim algoritma şu şekilde çalışacak. Algoritma her seferinde bir kelime tahmini yapacak ve o kelimeyi kullanarak elde ettiği bilgiyi, tüm sözlükte karşılaştırıp kelime setini bu bilgiye göre yeniden düzenleyecek. Bu algoritma temelde basit gibi dursa da eğer seçilecek kelimeyi rastgele belirlersem, bu en kötü durumda belki yüzlerce denemeyi gerekli kılabilir. Burdaki deneme sayısını optimize etmek ve oyun kuralları gereği 6 hamlede doğru cevabı bulabilmek için; kelime seçme algoritmasını doğru seçmek modelimizin en önemli kısmı. Kelime seçme problemini uygun şekilde çözebilmek için bu alanda geliştirilmiş farklı algoritmik yaklaşımları inceledim. İncelediğim algoritmalar şöyle;

Entropi / bilgi kazanımı (MaxEntropy / MaxInfo):
Bu yaklaşımın temel mantığı, bir tahminin ortalama olarak ne kadar bilgi üreteceğini ölçmektir. Her olası tahmin için, mevcut aday cevap kümesindeki her kelimeye karşı oluşabilecek geri bildirim desenleri (örneğin GYBBY gibi) hesaplanır. Aynı deseni üreten cevaplar bir gruplara (bucket) ayrılır. Bu grupların olasılık dağılımı üzerinden Shannon entropisi hesaplanır:

$$H = -\sum p_i \log_2 p_i$$

Burada p_i, i'inci geri bildirim deseninin olasılığıdır. Entropi ne kadar yüksekse, tahmin o kadar fazla bilgi üretir ve aday kümesini ortalama olarak daha iyi böler. Literatürde en güçlü klasik (ML

[1] Eğer gerekiyorsa ayrı bir sayfa kullanın

olmayan) yöntemlerden biri olarak kabul edilir ve genellikle 3–4 hamlede çözüm üretir

Entropi algoritmasını kullanan bir proje: https://github.com/ana-rosu/Wordle-Solver

Beklenen grup boyutu minimizasyonu (Min expected bucket size):
Bu yöntem entropiye çok benzer, ancak bilgi miktarını değil doğrudan bir sonraki adımda beklenen aday sayısını minimize etmeye çalışır.
Her tahmin için oluşabilecek tüm geri bildirim grupları hesaplanır ve şu değer bulunur:

$$E = \sum p_i \cdot |B_i|$$

Burada $|B_i|$, i'inci geri bildirim grubunun büyüklüğüdür. Amaç, tahmin sonrası kalan ortalama aday sayısını en aza indirmektir. Genelde entropi ile çok yakın sonuç verir.

Bu yaklaşımla yapılmış bir proje: https://github.com/CrispyConductor/wordle-solver

Minimax / largest bucket minimizasyonu:
Bu yaklaşım ortalama performans yerine en kötü durumu optimize eder. Her tahmin için oluşabilecek geri bildirim grupları hesaplanır ve en büyük gruba bakılır. Bu en büyük grup, "en kötü durumda" kaç aday kalacağını gösterir. Amaç fonksiyonu aşağıdaki gibidir;

$$\min_{guess} \max_{i} |B_i|$$

Bu yaklaşım olabilecek en kötü sonuçta olabilecek en büyük grubun büyüklüğünü minimize etmeye odaklanır. Worst-case performansı entropiden daha güçlüdür. Ortalama hamlede entropiden daha yavaş olabilir.
Minimax yaklaşımı ile yapılmış bir proje: https://github.com/mckoss/wordle-guesser

Frekans/Bayes öncülü ile ağırlıklandırma:
Bu yaklaşımda tüm aday cevapların eşit olasılıklı olduğu varsayımı bırakılır. Çünkü gerçekte bazı kelimeler bazılarından daha büyük olasılıkla cevap olur. Bunun için;

- Kelime kullanım sıklıkları (corpus frekansı),
- Wordle'ın resmi cevap listesi istatistikleri,
- Harf/pozisyon frekansları

gibi öncül olasılıklar kullanılır.
Bu durumda entropi veya expected bucket hesapları şu şekilde değiştirilir:

$$p_i = \sum_{\alpha \in B_i} P(\alpha)$$

Yani grupların olasılığı artık kelime sayısına göre değil, ağırlıklı olasılığa göre hesaplanır. Bu yaklaşım daha doğal bir performans sağlar çünkü geçekte tüm kelimeler eşit olasılıklı değildir.

Bu yaklaşım ile yapılmış bir proje: https://github.com/ilibarra/wordle_solver

RL/ML:
Bu yaklaşımlarda kelime tahmini bir öğrenme problemi olarak modellenir.

- **Supervised learning**: Kelime tahmini bir sınıflandırma problemi gibi ele alınır.
- **Reinforcement Learning (RL)**: Ajan her tahminde ödül alır (örneğin daha az aday kalması veya doğru cevap bulması). Politika gradient, Q-learning veya deep RL yöntemleri uygulanabilir.
- **Neural policy modelleri**: Girdi olarak mevcut durum (harf kısıtları) verilir, çıktı olarak tahmin kelimesi üretilir.

RL/ML yaklaşımları genelde deterministik yaklaşımlardan daha başarısızlar.

Yaptığım literatür taraması neticesinde, proje için hem implementasyon kolaylığı hem de algoritmanın başarısı sebebiyle en uygun tercihin entropi algoritması olduğuna karar verdim. Projemde bu algoritmayı kullandım.

4. Projede kullanılan veriler, dış bağlantılar hakkında bilgiler, verilerin tamamını veya birkaç örneği gösterebilirsiniz.

Projede iki temel veri kümesi kullanılmaktadır:

1. Cevap Kelimeleri Listesi (Answer List): Wordle'ın resmi 5 harfli cevap listesi (≈ 2300 kelime)
2. Tahmin Kelimeleri Listesi (Guess List): Tüm geçerli 5 harfli İngilizce kelimeler (≈ 12.000+ kelime)

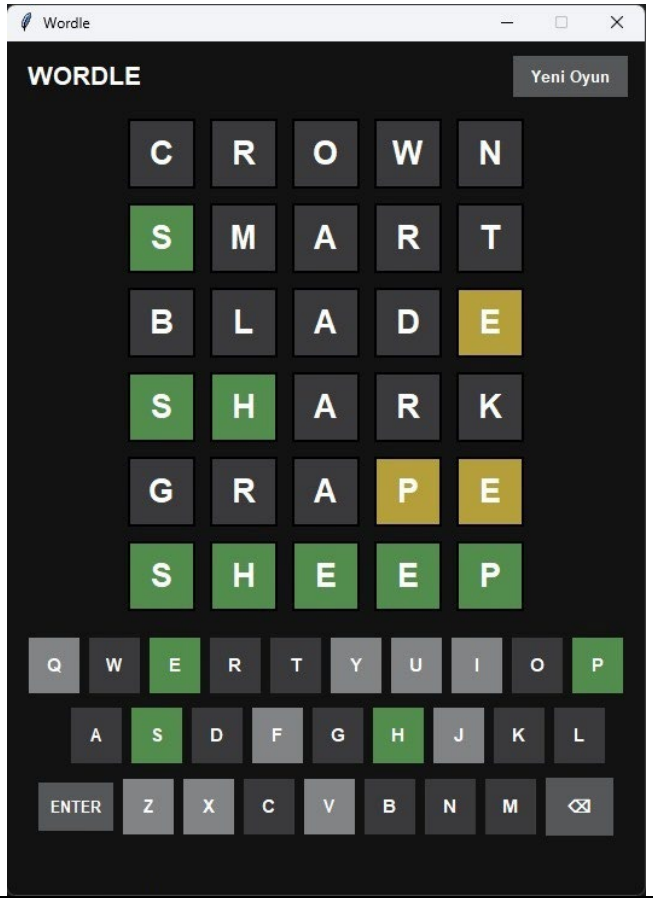
Veriler herhangi bir dış API gerektirmeden lokal olarak işlenmektedir.

5. Projenin ayrıntılı planı, akış şeması, veri işleyişi, vs, projede karşılaşılan sorunlar, projenin şu andaki durumu (varsa prototip, veya ekran çıktıları), bundan sonra yapılacaklar

Proje planım iki temel adımdan oluşuyor. İlk adım Wordle oyun motorunun python ile programlanması, ikinci adım ise entropi algoritması kullanılarak wordle solver modelinin python ile geliştirilmesi. İlk aşama 2 Mart itibarı ile tamamlanmış durumda. Projenin ikinci kısmı henüz başlanmadı. Projenin ikinci kısmı için planladığım bitiş tarihi 1 Nisan.

Proje Birinci Aşama:

- Wordle Oyun Motoru
  - Proje iskeleti, genel mimarinin oluşturulması
  - Oyun loopunun ve state sisteminin implementasyonu
  - Wordle geri bildirim kurallarının programlanması
  - Büyük kelime sözlüklerinin programa yüklenmesi
  - Tkinter kütüphanesi kullanılarak UI tasarımlarının yapılması/UI inputların düzenlenmesi
  - Tahmin ızgarasının ve ekran klavyesinin geri bildirimlere göre renklendirilmesi
  - Oyun koşullarının yönetimi ve kullanıcıya bildirimi/status bar
- Literatür Taraması
  - GitHub gibi kaynaklardan daha önce yapılmış wordle projelerinin ve worlde solver projelerinin toplanması ve incelenmesi
  - Wordle Solver algoritmaları için kullanılan algoritmaların araştırılması
  - Algoritmalardan kendi projem için en uygun olanın seçilip seçilen algoritma üzerinde ileri araştırma yapılması



Yanda eklediğim görsel. Projenin mevcut son halini göstermekte. Proje birinci aşamasında belirtilen tüm görevler tamamlanmış durumda, henüz 2. aşama olan yapay zeka modelinin geliştirilmesine başlanmadı. Şuan wordle oyunu oynanabilir durumda.

Proje İkinci Aşama:

- Worlde Solver Algoritması
  - Entropi hesaplama modülünün geliştirilmesi
  - Bucket oluşturma sistemi
  - Oyun ve algoritma arasında uygun iletişimin sağlanması
  - Tahmin optimizasyonu
  - Performans analizi

Planlanan Algoritma Akışı:

1. Başlangıçta tüm cevap listesi aday küme olarak yüklenecek
2. İlk tahmin statik bir tahmin olacak, bunun için en yüksek başarı oranı gösterdiği daha önceden hesaplanmış uygun kelime kullanılacak
3. Geri bildirim alınacak
4. Aday küme geri bildirim doğrultusunda filtrelenecek
5. Elde edilen bilgiler kullanılarak yeni aday kümesinde entropi hesaplanacak
6. Geri bildirim ve aday kümenin daraltılması işlemi tekrar edilecek
7. 1 aday kalana kadar bu döngü işleticek
8. Kalan son aday doğru tahmini verir

Veri işleyişi (Entropi algoritması nasıl implement edilecek):

Öncelikle mevcut kelime kümesi önceki tahminlerden elde edilen bilgiler kullanılarak belirlenir. Bu küme sadece cevap olabilecek kelimeleri içerir, geri bildirimlere uymayan kelimeler filtrelenmiştir.

Daha sonra tahmin havuzundaki her kelime tek tek değerlendirilir. Her tahmin kelimesi için şu işlem yapılır:

1. Mevcut aday kümedeki her kelime ile karşılaştırma yapılır.
2. Her karşılaştırma sonucunda Wordle kurallarına uygun bir geri bildirim deseni (G/Y/B kombinasyonu) üretilir.
3. Aynı geri bildirim desenini üreten kelimeler aynı grupta (bucket) toplanır.

Bu işlem sonucunda, ilgili tahmin kelimesi için "olası geri bildirim senaryolarına göre" aday kümenin nasıl bölüneceği simüle edilmiş olur.
Ardından her grubun büyüklüğü hesaplanır. Grup büyüklükleri kullanılarak her geri bildirim deseninin gerçekleşme olasılığı bulunur (grup boyutu / toplam aday sayısı).
Son aşamada Shannon entropi formülü uygulanır:

- Her grubun olasılığı ile logaritmik değeri çarpılır,
- Tüm gruplar için bu değerler toplanır,
- Negatif işaret uygulanarak ilgili tahmin kelimesinin entropi değeri elde edilir.

Bu işlem tüm olası tahmin kelimeleri için tekrar edilir. Sonuç olarak her kelime için sayısal bir "bilgi değeri" elde edilmiş olur.
En yüksek entropi değerine sahip kelime, aday kümesini en dengeli şekilde bölen ve ortalama olarak en fazla bilgi üreten tahmin olarak seçilir.
Gerçek geri bildirim alındıktan sonra ise aday küme şu şekilde güncellenir:

- Mevcut tahmin için üretilen geri bildirim desenini oluşturabilecek kelimeler korunur,
- Diğer tüm kelimeler elenir.

Bu sayede bilgi her turda sistematik olarak işlenmekte ve aday küme matematiksel olarak daraltılmaktadır.

Wordle Oyun Motorunu Geliştirirken Karşılaşılan Sorunlar

- Tekrarlı Harf İçeren Kelimelerde Geri Bildirim: Wordle oyununu yazarken karşılaştığım tek zorlayıcı problem buydu. Tahmin kelimesindeki harfleri, sadece ana kelimenin o harfi içerip içermediğine göre değerlendiremiyoruz. Çünkü sorulan kelimede kaç kez içerdiği de önemli. Mesela tahmin kelimesi bir harfi iki kez içerirken ana kelime bir kez içeriyorsa bu fazla harf sarı boyanabiliyor. Ama asıl kurala göre tek bir harfin sarı ya da yeşil boyanması gerekir. Yani hem harf sayısı hem de fazla harfin hangi sırada olduğu önemli(eğer sarı olan önce boyanırsa yeşil olması gereken gri olmak zorunda kalabilir). Bir harfin rengine karar verirken listedeki diğer tüm harfleri de göz önüne almak gerekiyor. Bu problemi çözmek için harf listeleriyle eşleştirilmiş int arrayler kullandım. Renkler ile ilgili karar verilmeden önce bu harfden kaç kez listede geçtiği kontrol ediliyor, eğer değer 1den farklı ise diğer harfin yeşil olup olmama ihtimali göz önünde bulundurularak renk ataması yapılıyor. Yapılan her atamada int array güncelleniyor.

Bundan Sonra Yapılacaklar

- Entropi modülünün implementasyonu
- Ortalama deneme sayısı analizi
- Sonuçların grafiksel analizi
- Alternatif algoritmalarla karşılaştırma

Planlanan optimizasyonlar:

- Ön hesaplama (precomputation)
- Aday küme küçüldükçe hesaplama alanını daraltma